

# Technical Note: block-DDA\_Py

Discrete Dipole Approximation with FFT-accelerated  
Block-Krylov Solver for Light Scattering by Arbitrary Particles

N. Moteki

Last updated: 2026-03-28 (corresponds to block-DDA\_Py v0.7.1)

## Contents

|          |                                                                       |           |
|----------|-----------------------------------------------------------------------|-----------|
| <b>1</b> | <b>Introduction</b>                                                   | <b>3</b>  |
| <b>2</b> | <b>DDA Formulation</b>                                                | <b>3</b>  |
| 2.1      | Fundamental equation . . . . .                                        | 3         |
| 2.2      | Incident field . . . . .                                              | 4         |
| 2.3      | Free-space dyadic Green's function . . . . .                          | 4         |
| 2.4      | Polarisability: Clausius–Mossotti with radiative correction . . . . . | 5         |
| <b>3</b> | <b>FFT-Accelerated Matrix–Vector Product (Goodman 1991)</b>           | <b>5</b>  |
| 3.1      | Interaction tensor on the doubled grid . . . . .                      | 5         |
| 3.2      | Pre-computation of the FFT . . . . .                                  | 6         |
| 3.3      | Block MVP via FFT convolution . . . . .                               | 6         |
| 3.4      | Computational complexity and memory . . . . .                         | 6         |
| 3.5      | Multi-threaded FFT . . . . .                                          | 7         |
| <b>4</b> | <b>Block-Krylov Iterative Solvers</b>                                 | <b>7</b>  |
| 4.1      | Jacobi preconditioning . . . . .                                      | 7         |
| 4.2      | Block-BiCGSTAB . . . . .                                              | 7         |
| 4.3      | Block-COCG-rq (alternative solver) . . . . .                          | 8         |
| <b>5</b> | <b>Computational Performance: Old vs. New Implementation</b>          | <b>8</b>  |
| 5.1      | MVP complexity . . . . .                                              | 8         |
| 5.2      | Parallelisation overhead . . . . .                                    | 8         |
| 5.3      | Worked example . . . . .                                              | 8         |
| 5.4      | Memory comparison . . . . .                                           | 9         |
| <b>6</b> | <b>Scattering Observables</b>                                         | <b>9</b>  |
| 6.1      | Absorption and extinction cross sections . . . . .                    | 9         |
| 6.2      | Forward scattering amplitude (PCAS) . . . . .                         | 10        |
| 6.3      | Backward scattering amplitude (OCBS) . . . . .                        | 10        |
| <b>7</b> | <b>Gaussian Random Ellipsoid (GRE) Shape Model</b>                    | <b>10</b> |
| 7.1      | Base ellipsoid . . . . .                                              | 11        |
| 7.2      | Surface deformation . . . . .                                         | 11        |
| 7.3      | Lattice generation and interior detection . . . . .                   | 11        |
| 7.4      | Volume-preserving lattice rescaling . . . . .                         | 11        |

|           |                                                        |           |
|-----------|--------------------------------------------------------|-----------|
| <b>8</b>  | <b>Spheroid Symmetry and Orientation Averaging</b>     | <b>12</b> |
| 8.1       | Euler angle mapping . . . . .                          | 12        |
| 8.2       | Constraint relations at forward scattering . . . . .   | 12        |
| 8.3       | Analytical phi-averaging (labor-saving mode) . . . . . | 12        |
| <b>9</b>  | <b>Spheroid Parameter Sweep</b>                        | <b>13</b> |
| 9.1       | Parameter grid . . . . .                               | 13        |
| 9.2       | Orientation grid on reduced domain . . . . .           | 14        |
| 9.3       | HDF5 output structure . . . . .                        | 14        |
| <b>10</b> | <b>Mie Theory Reference</b>                            | <b>14</b> |
| 10.1      | Partial-wave expansion . . . . .                       | 15        |
| 10.2      | Efficiencies and scattering amplitudes . . . . .       | 15        |

# 1 Introduction

The discrete dipole approximation (DDA) is a flexible numerical method for computing electromagnetic scattering and absorption by particles of arbitrary shape and composition [1, 2]. The target particle is represented by an array of  $N$  point dipoles on a cubic lattice with spacing  $d$ . Each dipole acquires a polarisation in response to the local electric field, which consists of the incident wave plus the fields radiated by all other dipoles.

This document describes the theoretical background and the algorithms that are implemented in version 0.4.0 of `block-DDA.Py`, covering:

1. the DDA formulation and polarisability prescription (Sec. 2),
2. the Goodman (1991) FFT-accelerated matrix–vector product (Sec. 3),
3. the block-Krylov iterative solvers (Sec. 4),
4. the scattering observables computed from the solution (Sec. 6),
5. the Gaussian Random Ellipsoid (GRE) shape model (Sec. 7),
6. the spheroid parameter sweep and its HDF5 output format (Sec. 9),
7. the Mie theory reference solution for homogeneous spheres (Sec. 10).

Throughout this note, the electromagnetic equations are written in Gaussian (CGS) units; lengths are expressed in micrometers ( $\mu\text{m}$ ). Vectors are set in bold italic ( $\mathbf{E}$ ,  $\mathbf{k}$ ,  $\mathbf{r}$ ), matrices and second-rank tensors in upright bold ( $\mathbf{A}$ ,  $\boldsymbol{\alpha}$ ), and unit vectors carry a hat ( $\hat{\mathbf{e}}$ ,  $\hat{\mathbf{r}}$ ).

## 2 DDA Formulation

### 2.1 Fundamental equation

Consider a monochromatic plane wave of vacuum wavelength  $\lambda_0$  incident on a particle embedded in a non-absorbing medium of refractive index  $m_m$ . The wavenumber in the medium is

$$k = \frac{2\pi m_m}{\lambda_0}. \quad (1)$$

The particle is discretised into  $N$  cubic elements (dipoles) of side length  $d$  and volume  $v = d^3$ , placed on a regular lattice with grid dimensions  $N_x \times N_y \times N_z$ . The lattice position of the  $i$ -th dipole is  $\mathbf{r}_i$ .

The polarisation  $\mathbf{P}_i$  of dipole  $i$  satisfies

$$\mathbf{P}_i = \boldsymbol{\alpha}_i \cdot \mathbf{E}(\mathbf{r}_i), \quad (2)$$

where  $\boldsymbol{\alpha}_i$  is the  $3 \times 3$  polarisability tensor and  $\mathbf{E}(\mathbf{r}_i)$  is the total electric field at position  $\mathbf{r}_i$ . The total field is the superposition of the incident field and the contributions from all other dipoles:

$$\mathbf{E}(\mathbf{r}_i) = \mathbf{E}_{\text{inc},i} - \sum_{j \neq i} \mathbf{G}(\mathbf{r}_i - \mathbf{r}_j) \cdot \mathbf{P}_j, \quad (3)$$

where  $\mathbf{G}$  is the free-space dyadic Green’s function (Sec. 2.3). Substituting (3) into (2) and rearranging yields the  $3N \times 3N$  linear system

$$\mathbf{A} \mathbf{P} = \mathbf{E}_{\text{inc}}, \quad (4)$$

where  $\mathbf{P}$  and  $\mathbf{E}_{\text{inc}}$  are column vectors of length  $3N$  formed by concatenating the three Cartesian components of each dipole (ordering:  $P_{x,0}, P_{y,0}, P_{z,0}, P_{x,1}, \dots$ ), and  $\mathbf{A}$  is a complex-valued matrix with

$$\mathbf{A}_{ij} = \begin{cases} \alpha_i^{-1} & \text{if } i = j, \\ \mathbf{G}(\mathbf{r}_i - \mathbf{r}_j) & \text{if } i \neq j. \end{cases} \quad (5)$$

When the dipoles are placed on a regular cubic lattice, the off-diagonal blocks  $\mathbf{A}_{ij}$  depend only on the difference  $\mathbf{r}_i - \mathbf{r}_j$ , giving  $\mathbf{A}$  a block-Toeplitz structure that can be exploited by FFT (Sec. 3).

## 2.2 Incident field

The incident plane wave in the medium is

$$\mathbf{E}_{\text{inc},i} = \mathbf{E}_0 \exp(i\mathbf{k} \cdot \mathbf{r}_i - i\omega t), \quad (6)$$

where  $\mathbf{k} = k \hat{\mathbf{u}}_{\text{inc}}$  with  $\hat{\mathbf{u}}_{\text{inc}}$  the propagation direction (fixed to  $+z$  in the laboratory frame). The time-harmonic factor  $e^{-i\omega t}$  is suppressed hereafter.

To simulate orientation-averaged scattering, the particle is rotated through  $L = N_\alpha \times N_\beta \times N_\gamma$  orientations on a deterministic grid of intrinsic ZYZ Euler angles:

- $\alpha$ :  $N_\alpha$  equally spaced values in  $[0, 2\pi)$ ,
- $\beta$ :  $N_\beta$  values chosen so that  $\cos \beta$  is equally spaced in  $(-1, 1)$  (equal-area midpoints on the sphere),
- $\gamma$ :  $N_\gamma$  equally spaced values in  $[0, 2\pi)$ .

This deterministic scheme avoids sampling noise and ensures reproducibility. For each orientation, the laboratory-frame vectors are transformed into the particle frame via the inverse rotation matrix. Batch rotation is performed using `scipy.spatial.transform.Rotation`.

Following the CAS-v2 (Complex Amplitude Sensing, version 2) beam configuration, the incident polarisation is assumed to be left-handed circular. This setting is hard-coded in the current version (v0.6.0). The polarisation vector is:

$$\hat{\mathbf{e}}_0 = \frac{1}{\sqrt{2}} \hat{\mathbf{e}}_\theta + \frac{i}{\sqrt{2}} \hat{\mathbf{e}}_\phi, \quad (7)$$

where  $\hat{\mathbf{e}}_\theta$  and  $\hat{\mathbf{e}}_\phi$  are the polar and azimuthal unit vectors associated with the incident direction.

## 2.3 Free-space dyadic Green's function

The  $3 \times 3$  interaction tensor for two dipoles separated by  $\mathbf{r} = \mathbf{r}_i - \mathbf{r}_j$  ( $r = |\mathbf{r}|$ ,  $\hat{\mathbf{r}} = \mathbf{r}/r$ ) is [3]

$$\mathbf{G}(\mathbf{r}) = \frac{e^{ikr}}{r} \left[ k^2 (\hat{\mathbf{r}} \otimes \hat{\mathbf{r}} - \mathbf{I}_3) + \frac{1 - ikr}{r^2} (\mathbf{I}_3 - 3 \hat{\mathbf{r}} \otimes \hat{\mathbf{r}}) \right], \quad (8)$$

where  $\otimes$  denotes the outer (tensor) product and  $\mathbf{I}_3$  the  $3 \times 3$  identity matrix. The self-interaction ( $\mathbf{r} = \mathbf{0}$ ) is excluded from  $\mathbf{G}$  and handled separately via the diagonal  $\alpha_i^{-1}$  in (5).

## 2.4 Polarisability: Clausius–Mossotti with radiative correction

The relative permittivity of the  $i$ -th dipole along Cartesian axis  $c$  ( $c \in \{x, y, z\}$ ) is

$$\epsilon_{r,c} = \frac{m_{p,c}^2}{m_m^2}, \quad (9)$$

where  $m_{p,c}$  is the (generally complex) particle refractive index along axis  $c$ .

The static Clausius–Mossotti (CM) polarisability is

$$\alpha_{0,c} = \frac{3}{4\pi} \frac{\epsilon_{r,c} - 1}{\epsilon_{r,c} + 2} v, \quad (10)$$

where  $v = d^3$  is the dipole volume.

To improve accuracy, the CM polarisability is corrected for radiative reaction following Chaumet & Rahmani [4]. Define the equivalent-volume sphere radius

$$a = \left( \frac{3v}{4\pi} \right)^{1/3} \quad (11)$$

and the correction term

$$M = \frac{8\pi}{3} [(1 - ika) e^{ika} - 1]. \quad (12)$$

The dynamical (radiative-reaction corrected) polarisability is then

$$\alpha_c = \frac{\alpha_{0,c}}{1 - M \alpha_{0,c} / v}. \quad (13)$$

In general the particle may be optically anisotropic, so  $\boldsymbol{\alpha}_i = \text{diag}(\alpha_x, \alpha_y, \alpha_z)$ . For isotropic particles,  $m_{p,x} = m_{p,y} = m_{p,z}$  and the polarisability reduces to a scalar times  $\mathbf{I}_3$ .

## 3 FFT-Accelerated Matrix–Vector Product (Goodman 1991)

Direct evaluation of the matrix–vector product (MVP)  $\mathbf{A} \mathbf{P}$  requires  $O(N^2)$  operations and  $O(N^2)$  storage. Because the off-diagonal part of  $\mathbf{A}$  has block-Toeplitz structure on a regular cubic lattice, the MVP can be evaluated via 3D FFT in  $O(N \log N)$  time with  $O(N)$  storage [3].

### 3.1 Interaction tensor on the doubled grid

The physical cuboid lattice has dimensions  $N_x \times N_y \times N_z$ . The index difference along each axis  $\alpha$  ranges from  $-(N_\alpha - 1)$  to  $+(N_\alpha - 1)$ . To evaluate the linear convolution as a cyclic convolution via FFT, the lattice is extended to  $2N_x \times 2N_y \times 2N_z$  with the following index mapping.

For each extended-grid index  $I_\alpha \in \{0, 1, \dots, 2N_\alpha - 1\}$ , the physical integer displacement  $\Delta\alpha$  is

$$\Delta\alpha(I_\alpha) = \begin{cases} I_\alpha & 0 \leq I_\alpha \leq N_\alpha - 1 \quad (\text{positive displacements}), \\ 0 & I_\alpha = N_\alpha \quad (\text{Nyquist; zeroed out}), \\ I_\alpha - 2N_\alpha & N_\alpha + 1 \leq I_\alpha \leq 2N_\alpha - 1 \quad (\text{negative displacements}). \end{cases} \quad (14)$$

The physical displacement vector at extended-grid index  $(I_x, I_y, I_z)$  is  $\mathbf{r} = d(\Delta x, \Delta y, \Delta z)$ , and the interaction tensor  $\mathbf{G}(\mathbf{r})$  is computed via (8). Three special cases are set to zero:

- self-interaction at the origin:  $\mathbf{G}(\mathbf{0}) = \mathbf{0}$ ,
- Nyquist positions ( $I_\alpha = N_\alpha$  for any  $\alpha$ ): physically unreachable displacements.

The resulting array has shape  $(2N_x, 2N_y, 2N_z, 3, 3)$  and dtype `complex128`.

### 3.2 Pre-computation of the FFT

Before the iterative solver begins, the 3D discrete Fourier transform of  $\mathbf{G}$  along the spatial axes is computed once:

$$\tilde{\mathbf{G}} = \text{FFT}_{3\text{D}}[\mathbf{G}]. \quad (15)$$

$\tilde{\mathbf{G}}$  has the same shape  $(2N_x, 2N_y, 2N_z, 3, 3)$  and is stored in memory throughout the simulation. It is independent of the incident direction, polarisation, and Krylov iteration, so this computation is performed only once per  $(\lambda_0, m_m)$  pair.

### 3.3 Block MVP via FFT convolution

Given a block of  $L$  polarisation vectors stacked as columns in  $\mathbf{P}_{\text{block}}$  (shape  $3N_x N_y N_z \times L$ ), the off-diagonal MVP  $\mathbf{A}_{\text{off}} \mathbf{P}_{\text{block}}$  is computed as follows.

1. **Reshape**  $\mathbf{P}_{\text{block}}$  from  $(3N_x N_y N_z, L)$  to the 5D physical representation  $(N_x, N_y, N_z, 3, L)$ .
2. **Zero-pad** to the doubled grid:

$$\hat{\mathbf{P}}(I_x, I_y, I_z, c, \ell) = \begin{cases} \mathbf{P}(I_x, I_y, I_z, c, \ell) & \text{if } I_\alpha < N_\alpha \ \forall \alpha, \\ 0 & \text{otherwise.} \end{cases} \quad (16)$$

3. **Forward 3D FFT** along spatial axes:  $\tilde{\mathbf{P}} = \text{FFT}_{3\text{D}}[\hat{\mathbf{P}}]$ .
4. **Pointwise tensor product** in Fourier space:

$$\tilde{Y}_{i,\ell} = \sum_{j=1}^3 \tilde{G}_{ij} \tilde{P}_{j,\ell} \quad (\text{at each spatial frequency}), \quad (17)$$

i.e. a  $3 \times 3$  matrix times a  $3 \times L$  matrix at every grid point.

5. **Inverse 3D FFT**:  $\hat{\mathbf{Y}} = \text{FFT}_{3\text{D}}^{-1}[\tilde{\mathbf{Y}}]$ .
6. **Extract** the physical region  $\hat{\mathbf{Y}}[:, :N_y, :N_z, :, :]$  and flatten back to  $(3N_x N_y N_z, L)$ .

The full preconditioned MVP including the diagonal is

$$\mathbf{A} \mathbf{S} = \text{diag}(\mathbf{A}) \cdot \mathbf{S} + \mathbf{A}_{\text{off}} \mathbf{S}, \quad (18)$$

where  $\text{diag}(\mathbf{A}) = \boldsymbol{\alpha}^{-1}$  is applied element-wise and  $\mathbf{A}_{\text{off}}$  is evaluated via the FFT procedure above.

### 3.4 Computational complexity and memory

- **Time**: Each MVP costs  $O(N_{\text{cuboid}} \log N_{\text{cuboid}})$  where  $N_{\text{cuboid}} = N_x N_y N_z$ . For the block variant with  $L$  right-hand sides, the  $L$  columns are processed in a single batched FFT call.
- **Memory**: The dominant allocations are:
  - $\tilde{\mathbf{G}}$ :  $2N_x \times 2N_y \times 2N_z \times 9$  complex128 entries =  $1152 N_{\text{cuboid}}$  bytes (static, allocated once);
  - padded polarisation arrays (forward and inverse FFT):  $2N_x \times 2N_y \times 2N_z \times 3 \times L$  complex128 entries each =  $384 L N_{\text{cuboid}}$  bytes (two such arrays at peak).

Peak memory  $\approx (1152 + 768 L) N_{\text{cuboid}}$  bytes.

### 3.5 Multi-threaded FFT

The 3D FFTs are computed by `scipy.fft.fftn` with multi-threading. The default number of worker threads is  $\max(1, n_{\text{cpu}} - 2)$ , leaving two cores free for other processes. This can be overridden at runtime via the environment variable `DDA_FFT_WORKERS`.

## 4 Block-Krylov Iterative Solvers

The DDA system (4) is solved for  $L$  right-hand sides simultaneously. Stacking the  $L$  incident-field vectors as columns of a block matrix  $\mathbf{B}$  (shape  $3N \times L$ ), the problem becomes

$$\mathbf{A} \mathbf{X} = \mathbf{B}, \quad (19)$$

where  $\mathbf{X}$  (shape  $3N \times L$ ) contains the  $L$  solution vectors.

### 4.1 Jacobi preconditioning

A left Jacobi (diagonal) preconditioner is applied:

$$\mathbf{M}^{-1} = \text{diag}(\mathbf{A})^{-1} = \text{diag}(\alpha_{c,i}), \quad (20)$$

so that the preconditioned system becomes  $\mathbf{M}^{-1} \mathbf{A} \mathbf{X} = \mathbf{M}^{-1} \mathbf{B}$ .

### 4.2 Block-BiCGSTAB

The primary solver is Block-BiCGSTAB [5], which extends the stabilised bi-conjugate gradient method to block systems. It is suitable for general (non-symmetric) matrices and is the default in `block-DDA.Py`.

The algorithm is summarised in Table 1. Each iteration requires two block MVPs (lines 1 and 4). The default convergence parameters are  $\tau = 0.01$  and  $n_{\text{max}} = 25$ .

Table 1: Block-BiCGSTAB with Jacobi preconditioning (after Tadano et al. [5]).

|                                                                                                                                                                        |
|------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <b>Input:</b> $\mathbf{A}$ , $\mathbf{B}$ (shape $3N \times L$ ), $\mathbf{M} = \text{diag}(\mathbf{A})$ , tolerance $\tau$ , max iterations $n_{\text{max}}$ .        |
| <b>Initialise:</b> $\mathbf{X}_0 = \mathbf{0}$ , $\mathbf{R}_0 = \mathbf{M}^{-1} \mathbf{B}$ , $\mathbf{P}_0 = \mathbf{R}_0$ , $\tilde{\mathbf{R}}_0 = \mathbf{R}_0$ . |
| <b>For</b> $n = 0, 1, \dots, n_{\text{max}} - 1$ :                                                                                                                     |
| 1. $\mathbf{V}_n = \mathbf{M}^{-1} \mathbf{A} \mathbf{P}_n$                                                                                                            |
| 2. $\alpha_n = (\tilde{\mathbf{R}}_0^H \mathbf{V}_n)^{-1} \tilde{\mathbf{R}}_0^H \mathbf{R}_n$                                                                         |
| 3. $\mathbf{T}_n = \mathbf{R}_n - \mathbf{V}_n \alpha_n$                                                                                                               |
| 4. $\mathbf{Z}_n = \mathbf{M}^{-1} \mathbf{A} \mathbf{T}_n$                                                                                                            |
| 5. $\xi_n = \text{tr}(\mathbf{Z}_n^H \mathbf{T}_n) / \text{tr}(\mathbf{Z}_n^H \mathbf{Z}_n)$                                                                           |
| 6. $\mathbf{X}_{n+1} = \mathbf{X}_n + \mathbf{P}_n \alpha_n + \xi_n \mathbf{T}_n$                                                                                      |
| 7. $\mathbf{R}_{n+1} = \mathbf{T}_n - \xi_n \mathbf{Z}_n$                                                                                                              |
| 8. If $\ \mathbf{R}_{n+1}\  / \ \mathbf{M}^{-1} \mathbf{B}\  < \tau$ : stop, return $\mathbf{X}_{n+1}$ .                                                               |
| 9. $\beta_n = (\tilde{\mathbf{R}}_0^H \mathbf{V}_n)^{-1} (-\tilde{\mathbf{R}}_0^H \mathbf{Z}_n)$                                                                       |
| 10. $\mathbf{P}_{n+1} = \mathbf{R}_{n+1} + (\mathbf{P}_n - \xi_n \mathbf{V}_n) \beta_n$                                                                                |

### 4.3 Block-COCG-rq (alternative solver)

Since the DDA interaction matrix  $\mathbf{A}$  is complex symmetric ( $\mathbf{A} = \mathbf{A}^\top$ ), a solver specialised for this structure, Block-COCG-rq [6], is also provided. Block-COCG-rq requires only one block MVP per iteration and is therefore faster per iteration than Block-BiCGSTAB. However, for particles with high refractive index it exhibits poorer convergence stability compared to Block-BiCGSTAB. To switch the solver, replace the import and function call in `bl_dda/scatterer.py` (line 3 and line 169): change `bl_bicgstab_jacobi_mvp_fft` to `bl_cocg_rq_jacobi_mvp_fft`. Both functions share the same interface.

## 5 Computational Performance: Old vs. New Implementation

This section compares the computational cost of the previous implementation (Barrowes 2001 block-Toeplitz FFT with `ray` multiprocessing) and the current implementation (Goodman 1991 3D FFT with NumPy/SciPy broadcasting).

### 5.1 MVP complexity

Both algorithms compute the off-diagonal matrix–vector product via FFT-based circular convolution, and both have the same asymptotic complexity per right-hand side (RHS):

$$T_{\text{MVP}}^{(1)} = O(N \log N),$$

where  $N = N_x N_y N_z$  is the total cuboid grid size. For  $L$  right-hand sides the total floating-point operation count is  $O(LN \log N)$  in both cases. The difference lies in execution efficiency.

### 5.2 Parallelisation overhead

Table 2: Execution-efficiency comparison (per Krylov iteration = two block MVPs for Block-BiCGSTAB).

|                                            | Old ( <code>ray</code> )                           | New (broadcasting)                                             |
|--------------------------------------------|----------------------------------------------------|----------------------------------------------------------------|
| Parallelism granularity                    | Process-level (1 MVP / worker)                     | Thread-level (FFT + BLAS)                                      |
| Inter-process communication                | Array serialise/deserialise                        | None (single process)                                          |
| MVP wall time ( $L$ RHS)                   | $\sim \frac{L}{P} T_{\text{MVP}} + T_{\text{ray}}$ | $\sim \frac{L}{W} T_{\text{FFT}} \times 2 + T_{\text{matmul}}$ |
| <code>ray</code> overhead $T_{\text{ray}}$ | $\sim 10\text{--}100$ ms / MVP                     | 0                                                              |
| Worker startup                             | $\sim 1\text{--}5$ s (pool init)                   | 0                                                              |
| Cache efficiency                           | Low (each worker holds copy)                       | High (contiguous memory)                                       |

Here  $P$  is the number of `ray` workers and  $W$  is the number of `scipy.fft` threads (default: `cpu_count - 2`).

### 5.3 Worked example

For  $N = 30\,000$ ,  $L = 50$ ,  $P = W = 8$ , and 10 Krylov iterations (20 MVPs):

The three main sources of speedup, in decreasing order of impact, are:

1. **Removal of `ray` IPC overhead.** Array serialisation/deserialisation is eliminated; the effect grows with  $N$ .
2. **Batched FFT.** All  $L$  RHS are processed in a single `scipy.fft` call; the internal thread pool distributes work efficiently.

Table 3: Estimated wall-time breakdown.

| Cost factor             | Old ( <b>ray</b> )                            | New (broadcasting)        |
|-------------------------|-----------------------------------------------|---------------------------|
| Effective MVP calls     | $20 \times \lceil 50/8 \rceil = 140$          | 20 (batched)              |
| <b>ray</b> IPC overhead | $\sim 140 \times 50 \text{ ms} = 7 \text{ s}$ | 0                         |
| Estimated speedup       | $1 \times$ (baseline)                         | $\sim 3\text{--}5 \times$ |

3. **Memory locality.** Contiguous-array access within a single process improves CPU cache hit rates.

## 5.4 Memory comparison

Table 4: Peak memory usage.

|                           | Old ( <b>ray</b> )                            | New (Goodman)                |
|---------------------------|-----------------------------------------------|------------------------------|
| Interaction tensor        | Copy per worker: $\sim P \times 1152 N$ bytes | Single copy: $1152 N$ bytes  |
| Polarisation (during MVP) | 1 RHS / worker                                | All $L$ RHS: $768 L N$ bytes |
| Total peak                | $\sim P \times 1152 N$                        | $(1152 + 768 L) \times N$    |

For  $P = 8$  and moderate  $L$ , the old implementation uses roughly  $8 \times$  more memory for the interaction tensor alone. The new implementation’s memory is dominated by the block polarisation arrays when  $L$  is large.

## 6 Scattering Observables

Once the polarisation  $\mathbf{P}_i$  ( $i = 1, \dots, N$ ) has been obtained for each orientation  $\ell$ , the following observables are computed. Throughout this section,  $k$  denotes the wavenumber in the medium as defined in (1).

### 6.1 Absorption and extinction cross sections

The absorption cross section is [2]

$$C_{\text{abs}} = 4\pi k \sum_{i=1}^N \text{Im}(\mathbf{P}_i \cdot \mathbf{E}_i^*), \quad (21)$$

where  $\mathbf{E}_i$  is the internal electric field at dipole  $i$ , reconstructed from the polarisation via

$$\mathbf{E}_i = \frac{4\pi \mathbf{P}_i}{(\epsilon_{r,i} - 1)v}. \quad (22)$$

The extinction cross section is

$$C_{\text{ext}} = 4\pi k \sum_{i=1}^N \text{Im}(\mathbf{P}_i \cdot \mathbf{E}_{\text{inc},i}^*). \quad (23)$$

The corresponding efficiency factors are  $Q_{\text{abs}} = C_{\text{abs}}/(\pi r_{ve}^2)$  and  $Q_{\text{ext}} = C_{\text{ext}}/(\pi r_{ve}^2)$ , where  $r_{ve}$  is the volume-equivalent sphere radius:

$$r_{ve} = \left( \frac{3 N v}{4\pi} \right)^{1/3}. \quad (24)$$

## 6.2 Forward scattering amplitude (PCAS)

The forward scattering amplitude in the PCAS (Polarimetric Complex Amplitude of Scattering) framework is computed from the far-field projection of the dipole polarisations.

For the forward scattering direction  $\hat{\mathbf{u}}_{\text{fw}} = +\hat{\mathbf{z}}$  in the laboratory frame, define the phase factor for each dipole:

$$e_{\text{fw},i} = \exp(ik \hat{\mathbf{u}}_{\text{fw}} \cdot \mathbf{r}_i). \quad (25)$$

The  $\theta$ - and  $\phi$ -components of the forward scattering amplitude correspond to the s- and p-polarisation channels of the CAS-v2 detector, respectively:  $S_{\text{fw},\theta}^{\text{PCAS}} \equiv S_s(0)$  and  $S_{\text{fw},\phi}^{\text{PCAS}} \equiv S_p(0)$ . They are computed as

$$S_{\text{fw},\theta}^{\text{PCAS}} [= S_s(0)] = k^2 \sum_{i=1}^N (\mathbf{P}_i \cdot \hat{\mathbf{e}}_{\theta,\text{fw}}) (\sqrt{2} e_{\text{fw},i})^*, \quad (26)$$

$$S_{\text{fw},\phi}^{\text{PCAS}} [= S_p(0)] = k^2 \sum_{i=1}^N (\mathbf{P}_i \cdot \hat{\mathbf{e}}_{\phi,\text{fw}}) (\sqrt{2} i e_{\text{fw},i})^*, \quad (27)$$

where the factors  $\sqrt{2}$  and  $\sqrt{2}i$  arise from the circular-polarisation decomposition of the incident field (7).

## 6.3 Backward scattering amplitude (OCBS)

For the exact backward scattering direction  $\hat{\mathbf{u}}_{\text{bk}} = -\hat{\mathbf{z}}$ , define the phase factor

$$e_{\text{bk},i} = \exp(ik \hat{\mathbf{u}}_{\text{bk}} \cdot \mathbf{r}_i). \quad (28)$$

Let  $\hat{\mathbf{e}}_{\theta,\text{bk}}$  and  $\hat{\mathbf{e}}_{\phi,\text{bk}}$  be the polar and azimuthal unit vectors associated with the backward direction. In the laboratory frame:  $\hat{\mathbf{e}}_{\theta,\text{bk}} = -\hat{\mathbf{x}}$  and  $\hat{\mathbf{e}}_{\phi,\text{bk}} = +\hat{\mathbf{y}}$ .

The  $\theta$ - and  $\phi$ -components of the backward scattering amplitude are

$$S_{\text{bk},\theta}^{\text{OCBS}} = k^2 \sum_{i=1}^N (\mathbf{P}_i \cdot \hat{\mathbf{e}}_{\theta,\text{bk}}) (\sqrt{2} e_{\text{bk},i})^*, \quad (29)$$

$$S_{\text{bk},\phi}^{\text{OCBS}} = k^2 \sum_{i=1}^N (\mathbf{P}_i \cdot \hat{\mathbf{e}}_{\phi,\text{bk}}) (\sqrt{2} i e_{\text{bk},i})^*, \quad (30)$$

where the reference-wave factors  $\sqrt{2}$  and  $\sqrt{2}i$  are the same as in the forward case.

The OCBS (Opposite-Circular-Basis Scattering) amplitude is then

$$S_{\text{bk}}^{\text{OCBS}} = \frac{-S_{\text{bk},\theta}^{\text{OCBS}} + S_{\text{bk},\phi}^{\text{OCBS}}}{\sqrt{2}}. \quad (31)$$

More precisely, the  $\theta$ - and  $\phi$ -components correspond to combinations of the Mishchenko amplitude matrix elements [9]:

$$S_{\text{bk},\theta}^{\text{OCBS}} \equiv S_{11}(\pi) + i S_{12}(\pi), \quad S_{\text{bk},\phi}^{\text{OCBS}} \equiv S_{22}(\pi) - i S_{21}(\pi). \quad (32)$$

The backscattering theorem  $S_{21}(\pi) = -S_{12}(\pi)$  causes the off-diagonal contributions to cancel when the two components are combined in (31), yielding an expression equivalent to Eq. (28) of Moteki and Adachi [10].

## 7 Gaussian Random Ellipsoid (GRE) Shape Model

The particle shape is generated using the Gaussian Random Ellipsoid (GRE) model of Muinonen & Pieniluoma [7].

## 7.1 Base ellipsoid

The base ellipsoid is defined by three semi-axes  $a \geq b \geq c$  and the volume-equivalent radius  $r_v$ . Two axis ratios are specified as input:  $b/c$  (`bc_ratio`) and  $a/b$  (`ab_ratio`). The semi-axis  $c$  is determined by volume conservation:

$$c = \left( \frac{r_v^3}{(a/b)(b/c)^2} \right)^{1/3}, \quad b = c \cdot (b/c), \quad a = b \cdot (a/b). \quad (33)$$

## 7.2 Surface deformation

A Gaussian random field  $s(\theta, \phi)$  is sampled on the surface of the base ellipsoid with covariance

$$\text{Cov}(s_i, s_j) = \beta^2 \exp\left(-\frac{\|\mathbf{r}_i - \mathbf{r}_j\|^2}{2\ell_c^2}\right), \quad (34)$$

where  $\beta$  is the standard deviation of the deformation and  $\ell_c$  is the correlation length, which controls the fineness of the surface roughness. In the current version (v0.6.0),  $\ell_c = 0.3c$  is adopted as an empirical choice.

The deformed surface position is obtained by displacing each point on the base ellipsoid along the outward surface normal  $\hat{\mathbf{n}}_0$ :

$$\mathbf{r}_{\text{GRE}}(\theta, \phi) = \mathbf{r}_{\text{base}}(\theta, \phi) + h_0 [\exp(s(\theta, \phi)) - \frac{1}{2}\beta^2 - 1] \hat{\mathbf{n}}_0(\theta, \phi), \quad (35)$$

where  $h_0 = c^2/a$  is a length scale and  $\hat{\mathbf{n}}_0$  is the unit outward normal to the base ellipsoid.

## 7.3 Lattice generation and interior detection

The lattice spacing is determined by the number of dipoles per wavelength inside the particle (dpl):

$$d = \frac{\lambda_{\text{particle,min}}}{\text{dpl}} = \frac{\lambda_0}{\max_c |m_{p,c}| \times \text{dpl}}, \quad (36)$$

where  $m_{p,c}$  ( $c \in \{x, y, z\}$ ) is the complex particle refractive index along axis  $c$  (cf. Sec. 2.4) and  $\lambda_{\text{particle,min}} = \lambda_0 / \max_c |m_{p,c}|$  is the shortest wavelength inside the particle. The default value is `dpl = 17`, which is a typical choice for DDA simulations of sub-micrometre particles. A cubic lattice with this spacing is generated within the bounding box of the deformed surface. Interior points are identified by a three-axis ray-casting algorithm.

## 7.4 Volume-preserving lattice rescaling

Because the GRE surface is discretised onto a cubic lattice, the total volume of the  $N$  occupied dipoles,  $Nd^3$ , generally differs from the target volume  $V_{\text{target}} = \frac{4}{3}\pi r_{v,\text{base}}^3$  by a few percent. When parameter sweeps require exact control of the volume-equivalent radius (e.g. an equally-spaced  $r_{v,\text{base}}$  grid), this discretisation error is undesirable.

To eliminate it, the lattice spacing is rescaled *after* the interior detection step. Given the number of occupied sites  $N$  and the desired  $r_{v,\text{base}}$ , the adjusted spacing is

$$d_{\text{adj}} = \left( \frac{V_{\text{target}}}{N} \right)^{1/3} = \left( \frac{4\pi}{3N} \right)^{1/3} r_{v,\text{base}}. \quad (37)$$

All lattice coordinates are then multiplied by the scale factor  $s = d_{\text{adj}}/d$ , and  $d_{\text{adj}}$  replaces  $d$  in every subsequent computation: the Green's function (8), the Clausius–Mossotti polarisability (10), and the radiative correction (13). Because both the inter-dipole distances and the dipole volume  $v = d_{\text{adj}}^3$  change by the same factor, the DDA formulation remains self-consistent.

This rescaling is performed in the `Target` constructor (`bl_dda/scatterer.py`), which requires `r_v_base` as a mandatory argument.

## 8 Spheroid Symmetry and Orientation Averaging

For spheroids (axially symmetric particles with  $a = b$  and GRE roughness  $\beta_{\text{GRE}} = 0$ ), the scattering amplitudes possess analytical constraint relations that enable significant computational savings in multi-orientation calculations.

### 8.1 Euler angle mapping

The intrinsic ZYZ Euler angles  $(\alpha, \beta, \gamma)$  relate to the spheroid geometry as follows:

- $\beta$ : polar angle of the symmetry axis (tilt from the laboratory  $z$ -axis),
- $\alpha$ : azimuthal angle of the symmetry axis (rotation of the tilt plane about  $z$ ),
- $\gamma$ : rotation about the symmetry axis; irrelevant for a spheroid ( $a = b$ ).

In the notation of Sec. 8.2,  $\theta \equiv \beta$  and  $\phi \equiv \alpha$ .

### 8.2 Constraint relations at forward scattering

When the symmetry axis lies in the  $xz$ -plane ( $\alpha = 0$ ), the cross-polarisation elements of the amplitude scattering matrix vanish:  $S_{12}(\theta, 0) = S_{21}(\theta, 0) = 0$ . The forward-scattering amplitudes at arbitrary  $\alpha$  are then determined entirely by the two diagonal elements at  $\alpha = 0$ :

$$S_s(\theta, \alpha) = \frac{S_{11}(\theta, 0) + S_{22}(\theta, 0)}{2} + \frac{S_{11}(\theta, 0) - S_{22}(\theta, 0)}{2} e^{i2\alpha}, \quad (38)$$

$$S_p(\theta, \alpha) = \frac{S_{11}(\theta, 0) + S_{22}(\theta, 0)}{2} - \frac{S_{11}(\theta, 0) - S_{22}(\theta, 0)}{2} e^{i2\alpha}, \quad (39)$$

where  $S_s \equiv S_{11} + iS_{12}$  and  $S_p \equiv S_{22} - iS_{21}$  are the PCAS observables (Sec. 6.2).

These imply three verifiable symmetries:

1. **Trace invariance:**  $S_s(\alpha) + S_p(\alpha) = S_{11}(\theta, 0) + S_{22}(\theta, 0) = \text{const.}$
2. **Modulus preservation:**  $|S_s(\alpha) - S_p(\alpha)| = |S_{11}(\theta, 0) - S_{22}(\theta, 0)| = \text{const.}$
3. **Cross-symmetry:**  $S_s(\theta, \alpha + \pi/2) = S_p(\theta, \alpha).$

These relations are verified numerically at machine precision ( $\sim 10^{-16}$ ) in `test_spheroid_symmetry.ipynb`.

### 8.3 Analytical phi-averaging (labor-saving mode)

For orientation-averaged observables, the azimuthal average over  $\alpha \in [0, 2\pi]$  eliminates the oscillatory term  $e^{i2\alpha}$ , yielding

$$\langle S_s(\theta) \rangle_\alpha = \langle S_p(\theta) \rangle_\alpha = \frac{S_{11}(\theta, 0) + S_{22}(\theta, 0)}{2}. \quad (40)$$

Furthermore,  $C_{\text{ext}}$  and  $C_{\text{abs}}$  are also  $\alpha$ -invariant for a spheroid, so the values computed at  $\alpha = 0$  are identical to their  $\alpha$ -averages.

**Implementation.** When `run_dda.py` detects a spheroid (`ab_ratio == 1` and `gre_beta == 0`), it activates *spheroid mode*:

- The DDA is solved only for  $N_\beta$  orientations at  $\alpha = 0$ ,  $\gamma = 0$ , with  $\cos\beta$  at the  $N_\beta$  equal-area midpoints (Sec. 2.2).

- The full  $N_\alpha \times N_\beta \times N_\gamma$  orientation grid is then filled analytically using the constraint relations (38)–(39):

$$S_s(\alpha_j, \beta_i) = A_i + B_i e^{i2\alpha_j}, \quad S_p(\alpha_j, \beta_i) = A_i - B_i e^{i2\alpha_j},$$

where  $A_i = [S_s(\beta_i, 0) + S_p(\beta_i, 0)]/2$  and  $B_i = [S_s(\beta_i, 0) - S_p(\beta_i, 0)]/2$ .

- For backward scattering, the OCBS amplitude follows the same rotational structure:  $S_{\text{bk}}^{\text{OCBS}}(\alpha_j, \beta_i) = S_{\text{bk}}^{\text{OCBS}}(0, \beta_i) e^{i2\alpha_j}$ .
- $C_{\text{ext}}$  and  $C_{\text{abs}}$  are  $\alpha$ -invariant, so the  $\alpha = 0$  values are broadcast to all grid points.

This reduces the number of DDA solves from  $N_\alpha \times N_\beta \times N_\gamma$  to  $N_\beta$ , while the output grid retains the full  $N_\alpha \times N_\beta \times N_\gamma$  resolution with *exact* (not interpolated) values at every grid point.

**Applicability.** These relations hold for any particle whose cross-section in the plane perpendicular to the symmetry axis is circular ( $a = b$ ), regardless of the axial ratio  $b/c$ . They break down for particles lacking a continuous rotational symmetry axis (e.g., tri-axial ellipsoids with  $a \neq b$ , or particles with GRE roughness  $\beta_{\text{GRE}} > 0$ ).

**Motivation.** For spheroids, the  $T$ -matrix method [9] is generally far more efficient than DDA. However, the extended-boundary-condition method (EBCM) used to compute the  $T$ -matrix becomes numerically unstable for extreme aspect ratios ( $b/c \ll 1$  or  $b/c \gg 1$ ), where the surface integrals over highly elongated or flattened geometries lead to severe loss of numerical precision [9]. In such regimes, DDA with the spheroid-mode optimisation described above provides a viable alternative: the computation remains stable regardless of the aspect ratio, and the analytical  $\alpha$ -averaging reduces the cost by a factor of  $N_\alpha$  relative to the general DDA mode.

## 9 Spheroid Parameter Sweep

For systematic studies of spheroid scattering properties, a dedicated sweep script (`run_dda_spheroid_sweep.py`) constructs equidistant grids over three physical parameters and a two-dimensional orientation grid on a reduced domain, executes the DDA in spheroid mode (Sec. 8), and stores the forward-scattering amplitudes in an HDF5 file. The output schema is specified in `spheroid_h5_schema_spec.md` and is compatible with the downstream LUT builder (`build_spheroid_lut.py`).

### 9.1 Parameter grid

The physical-parameter grid is spanned by three equidistant axes:

$$D_{ve} \in [D_{ve,\text{min}}, D_{ve,\text{max}}], \quad N_{D_{ve}} \text{ equally spaced points}, \quad (41)$$

$$n_{\text{real}} \in [n_{\text{real},\text{min}}, n_{\text{real},\text{max}}], \quad N_{\text{RI}} \text{ equally spaced points}, \quad (42)$$

$$\log_{10}(\text{AR}) \in [\log_{10} \text{AR}_{\text{min}}, \log_{10} \text{AR}_{\text{max}}], \quad N_{\text{AR}} \text{ equally spaced points}, \quad (43)$$

where  $D_{ve} = 2r_v$  is the volume-equivalent diameter,  $n_{\text{real}} = \text{Re}(m_p)$  is the real part of the isotropic particle refractive index, and  $\text{AR} = b/c$  is the aspect ratio of the spheroid (equal to `bc_ratio`;  $\text{AR} > 1$  oblate,  $< 1$  prolate).

The vacuum wavelength  $\lambda_0$  and medium refractive index  $m_m$  are fixed parameters (stored as HDF5 root attributes, not swept). The imaginary refractive index  $\text{Im}(m_p)$  is also fixed (default 0). The total number of DDA conditions is  $N_{D_{ve}} \times N_{\text{RI}} \times N_{\text{AR}}$ .

The aspect-ratio grid is equally spaced in  $\log_{10}(\text{AR})$  so that oblate and prolate shapes are sampled symmetrically about the sphere ( $\text{AR} = 1$ , i.e.  $\log_{10} \text{AR} = 0$ ).

## 9.2 Orientation grid on reduced domain

For spheroids, the body-frame rotation angle  $\gamma$  is irrelevant (Sec. 8). Furthermore, the forward-scattering amplitudes possess two symmetries that halve the required domain:

1. **Polar mirror symmetry:**  $\mathcal{S}(\cos \theta_o) = \mathcal{S}(-\cos \theta_o)$ , so only  $\cos \theta_o \in [0, 1]$  needs to be stored.
2. **Azimuthal periodicity:**  $\mathcal{S}(\theta_o, \phi_o) = \mathcal{S}(\theta_o, \phi_o + \pi)$ , so only  $\phi_o \in [0, \pi]$  needs to be stored.

The orientation grid is therefore:

$$\cos \theta_{o,j} = \frac{j}{N_u - 1}, \quad j = 0, \dots, N_u - 1, \quad (44)$$

$$\phi_{o,i} = \frac{\pi i}{N_\phi - 1}, \quad i = 0, \dots, N_\phi - 1, \quad (45)$$

where  $N_u$  is the number of grid points in  $\cos \theta_o \in [0, 1]$  and  $N_\phi$  is the number of grid points in  $\phi_o \in [0, \pi]$ . Both grids are equidistant and include both endpoints.

The orientation angles map to Euler angles as  $\theta_o \leftrightarrow \beta$  (polar) and  $\phi_o \leftrightarrow \alpha$  (azimuthal). As described in Sec. 8, only  $N_u$  DDA solves are performed (at  $\phi_o = 0$ ); the full  $(\cos \theta_o, \phi_o)$  grid is populated using the analytical constraint relations.

## 9.3 HDF5 output structure

Results are stored in `dda_results/dda_results_spheroid_sweep.h5`. All datasets reside in the HDF5 root group (no subgroups).

- **Grid axis arrays** (1D, equidistant):

| Dataset                            | Shape           | dtype   | Description                     |
|------------------------------------|-----------------|---------|---------------------------------|
| <code>D_ve_grid</code>             | $(N_{D_{ve}},)$ | float64 | Volume-equivalent diameter [μm] |
| <code>RI_real_grid</code>          | $(N_{RI},)$     | float64 | Real refractive index           |
| <code>log_AR_grid</code>           | $(N_{AR},)$     | float64 | $\log_{10}(\text{AR})$          |
| <code>cos_theta_o_half_grid</code> | $(N_u,)$        | float64 | $\cos \theta_o$ on $[0, 1]$     |
| <code>phi_o_grid</code>            | $(N_\phi,)$     | float64 | $\phi_o$ on $[0, \pi]$ [rad]    |

- **Forward scattering amplitudes** (shape  $(N_{D_{ve}}, N_{RI}, N_{AR}, N_u, N_\phi)$ , dtype float64): `S_fw_theta_re`, `S_fw_theta_im`, `S_fw_phi_re`, `S_fw_phi_im`. Non-converged cells contain NaN.
- **Convergence flag** (shape  $(N_{D_{ve}}, N_{RI}, N_{AR})$ , dtype bool): `converged`. True where the DDA solver converged.
- **Root attributes** (provenance): `wl_0`, `m_m`, `m_imag`, `rng_seed`, `solver_tol`, `block_dda_version`.

**Symmetry handling.** The HDF5 contains only the reduced-domain data. The downstream consumer mirrors  $\cos \theta_o$  to  $[-1, 1]$  and extends  $\phi_o$  to  $[0, 2\pi)$ . The analytical expansion from  $\beta$ -only DDA solves to the full  $(\cos \theta_o, \phi_o)$  grid is performed by `block-DDA-Py` before writing.

A worked example for reading a single data point is provided in `read_spheroid_sweep_example.py`.

## 10 Mie Theory Reference

For validation, the DDA solution for a volume-equivalent sphere is compared to the exact Mie theory [8].

## 10.1 Partial-wave expansion

For a homogeneous sphere of radius  $r_p$ , the size parameter is  $x = k r_p = 2\pi m_m r_p / \lambda_0$  and the relative refractive index is  $m_r = m_p / m_m$ . The number of partial-wave terms is

$$n_{\max} = \lfloor |x + 4x^{1/3} + 2| \rfloor. \quad (46)$$

The Mie coefficients  $a_n$  and  $b_n$  are computed from Riccati–Bessel functions  $\psi_n$ ,  $\chi_n$ ,  $\xi_n = \psi_n + i\chi_n$  and the logarithmic derivative  $D_n(m_r x)$  via downward recurrence [8].

## 10.2 Efficiencies and scattering amplitudes

$$Q_{\text{ext}} = \frac{2}{x^2} \sum_{n=1}^{n_{\max}} (2n+1) \operatorname{Re}(a_n + b_n), \quad (47)$$

$$Q_{\text{sca}} = \frac{2}{x^2} \sum_{n=1}^{n_{\max}} (2n+1) (|a_n|^2 + |b_n|^2), \quad (48)$$

$$Q_{\text{abs}} = Q_{\text{ext}} - Q_{\text{sca}}. \quad (49)$$

The angular scattering amplitudes are [8]

$$S_1(\theta) = \sum_{n=1}^{n_{\max}} \frac{2n+1}{n(n+1)} (a_n \pi_n(\cos \theta) + b_n \tau_n(\cos \theta)), \quad (50)$$

$$S_2(\theta) = \sum_{n=1}^{n_{\max}} \frac{2n+1}{n(n+1)} (a_n \tau_n(\cos \theta) + b_n \pi_n(\cos \theta)), \quad (51)$$

where  $\pi_n$  and  $\tau_n$  are the angular functions computed via upward recurrence. The forward and backward scattering amplitudes used for validation are obtained from  $S_1$  and  $S_2$  at  $\theta = 0$  and  $\theta = \pi$ .

## References

- [1] E. M. Purcell and C. R. Pennypacker, “Scattering and absorption of light by nonspherical dielectric grains,” *Astrophys. J.* **186**, 705–714 (1973).
- [2] B. T. Draine, “The discrete-dipole approximation and its application to interstellar graphite grains,” *Astrophys. J.* **333**, 848–872 (1988).
- [3] J. J. Goodman, B. T. Draine, and P. J. Flatau, “Application of fast-Fourier-transform techniques to the discrete-dipole approximation,” *Opt. Lett.* **16**(15), 1198–1200 (1991).
- [4] P. C. Chaumet and A. Rahmani, “Coupled-dipole method for magnetic and negative-refraction materials,” *J. Quant. Spectrosc. Radiat. Transfer* **110**, 22–29 (2009).
- [5] H. Tadano, T. Sakurai, and Y. Kuramashi, “Block BiCGSTAB( $\ell$ ) for linear systems with multiple right-hand sides,” *JSIAM Lett.* **1**, 44–47 (2009).
- [6] X.-M. Gu, T.-Z. Huang, L. Li, H.-B. Li, T. Sogabe, and M. Clemens, “Block variants of COCG and COCR methods for solving complex symmetric linear systems with multiple right-hand sides,” arXiv:1611.09813 (2016).
- [7] K. Muinonen and T. Pieniluoma, “Light scattering by Gaussian random ellipsoid particles: First results with discrete-dipole approximation,” *J. Quant. Spectrosc. Radiat. Transfer* **112**, 1747–1752 (2011).

- [8] C. F. Bohren and D. R. Huffman, *Absorption and Scattering of Light by Small Particles* (Wiley, New York, 1983).
- [9] M. I. Mishchenko, L. D. Travis, and A. A. Lacis, *Scattering, Absorption, and Emission of Light by Small Particles* (Cambridge University Press, 2002).
- [10] N. Moteki and K. Adachi, “Measuring the polarized complex forward-scattering amplitudes of single particles in unbounded fluid flow: CAS-v2 protocol,” *Opt. Express* **32**(21), 36500–36522 (2024). <https://doi.org/10.1364/OE.533776>.

## Acknowledgment

This document was prepared with the assistance of Claude (Anthropic). The author assumes full responsibility for the content.